

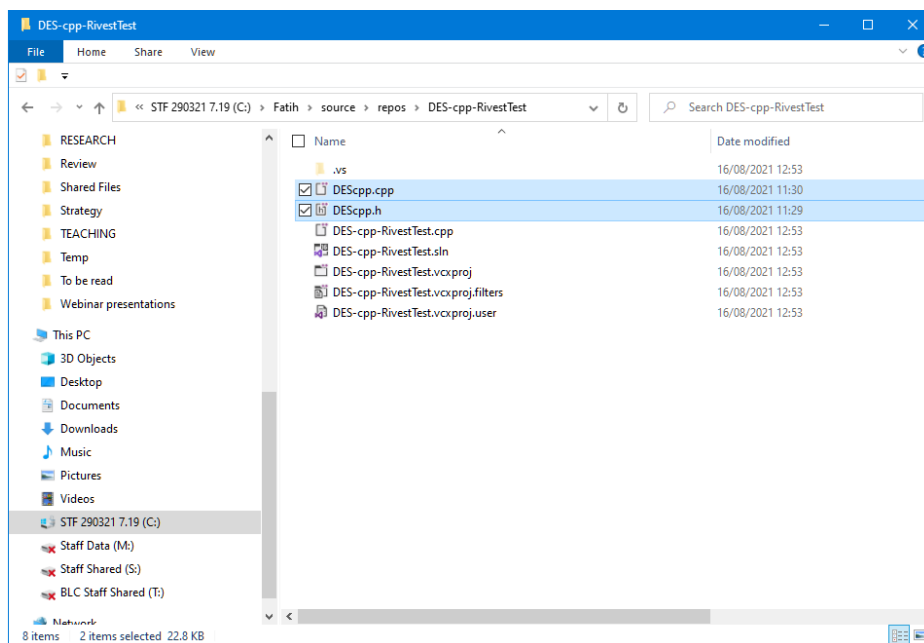
Lab Exercise 03: Validation of DES Implementation

BACKGROUND:

We discussed the DES algorithm in the classes and how we can use an open source implementation in our security projects. In this Lab, we will investigate how we can test and deploy a DES implementation. We want to use an open source C++ implementation, `DEScpp`, in our projects. Before using it, first, we need to understand the interfaces for this implementation and test the implementation to find out if it was implemented DES algorithm correctly. For this aim, we will use the Rivest Test which we discussed in our Tutorial.

PREPARATION:

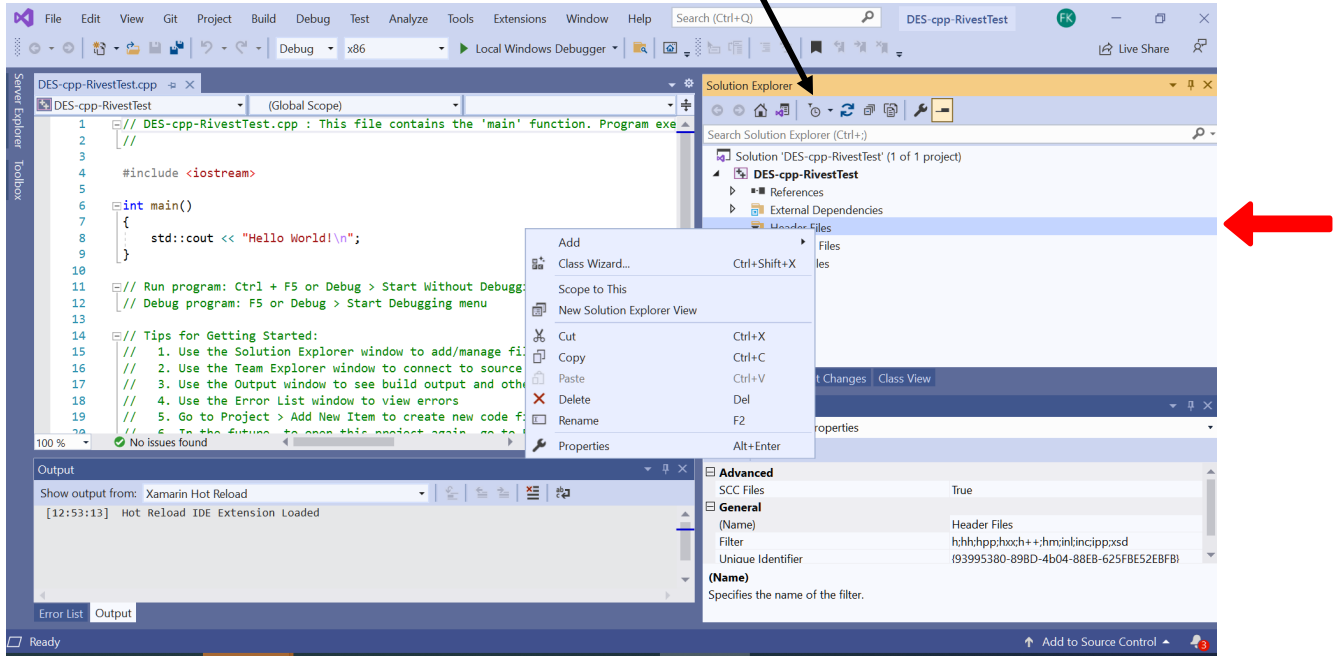
1. Download `DEScpp.cpp` and `DEScpp.h` to your `repos` folder from the **Course Resources**.
2. Run Visual Studio 2019 and create a new project named “DES-cpp-RivestTest”.
Hint: You can refer to the previous Lab documents to see how you can create a new project.
3. Using Windows File Explorer, copy `DEScpp.cpp` and `DEScpp.h` to the solution folder for DES-cpp-RivestTest created by Visual Studio. DES-cpp-RivestTest folder should be as follows:



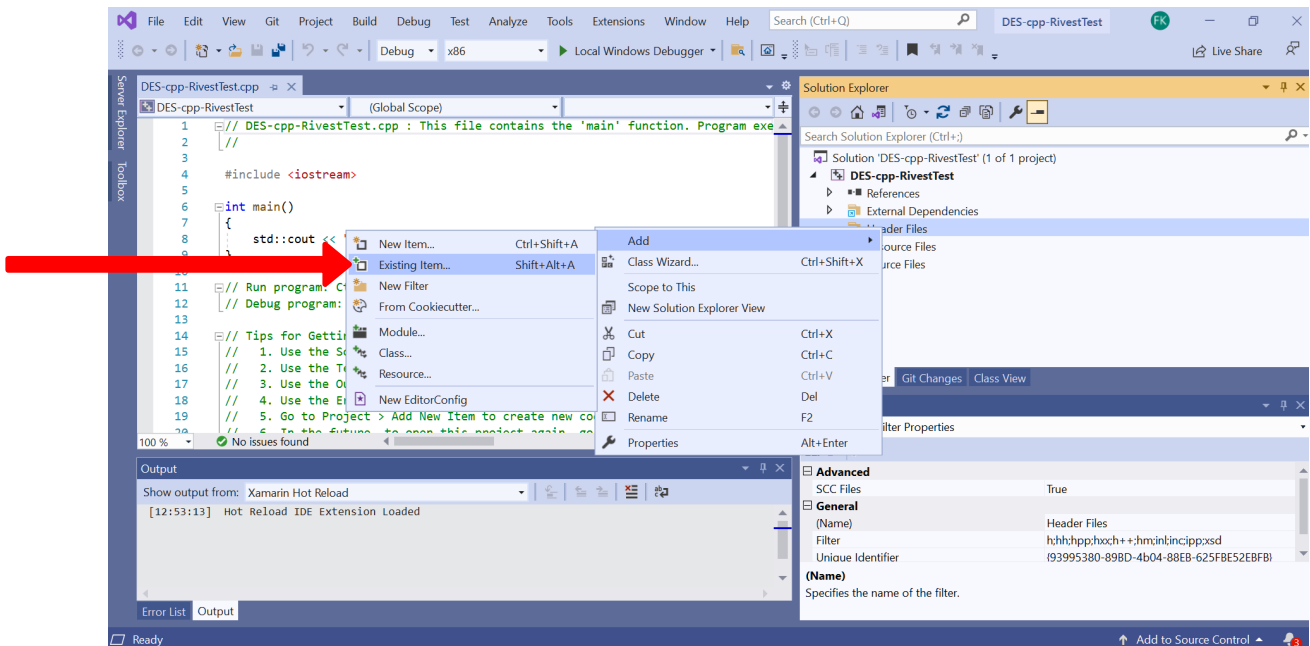
Open source files
copied to this folder
by the user.

Files created by the
Visual Studio.

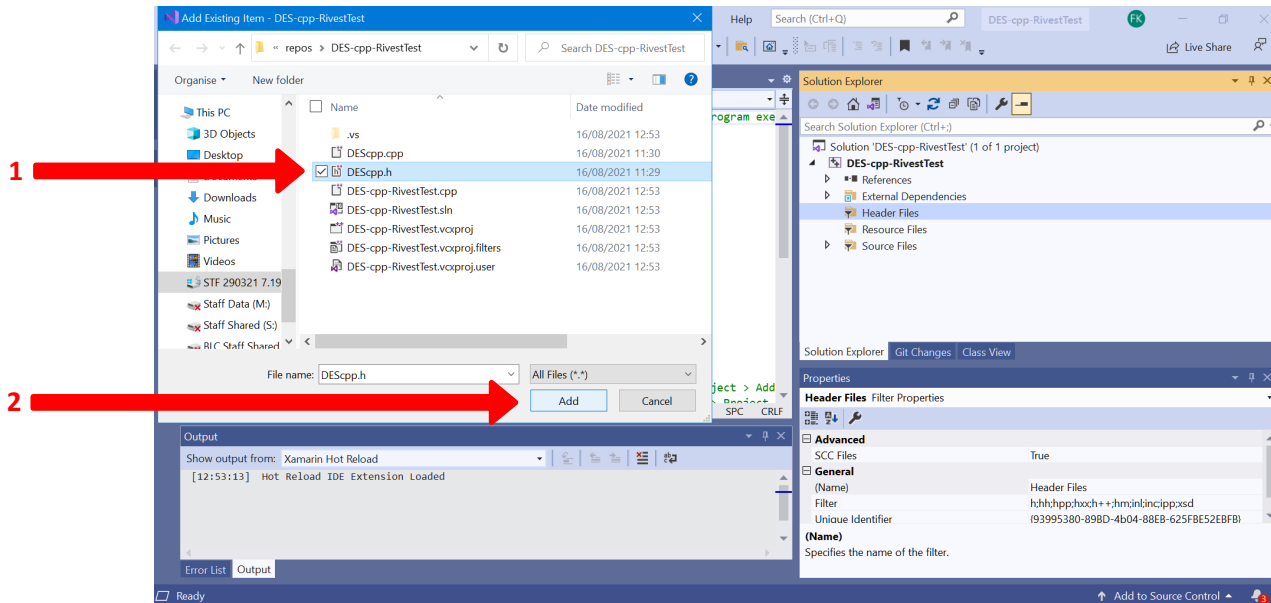
- Now we need to add these files to our project. Go back to Visual Studio.
- To add the header file, `DESspp.h`, go to the Solution Explorer window and hover over Header Files. A right click on Header Files will open a Menu:



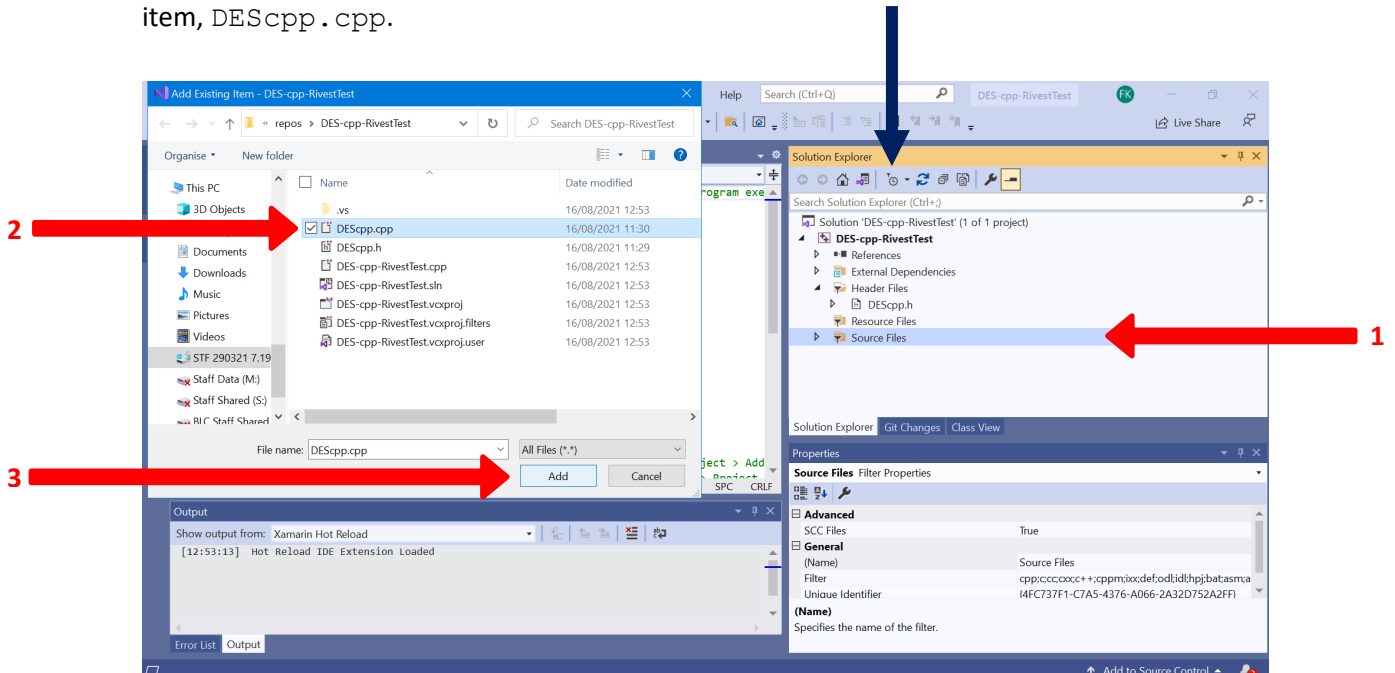
- Hover over 'Add item'. This will open another window. Click on 'Existing Item':



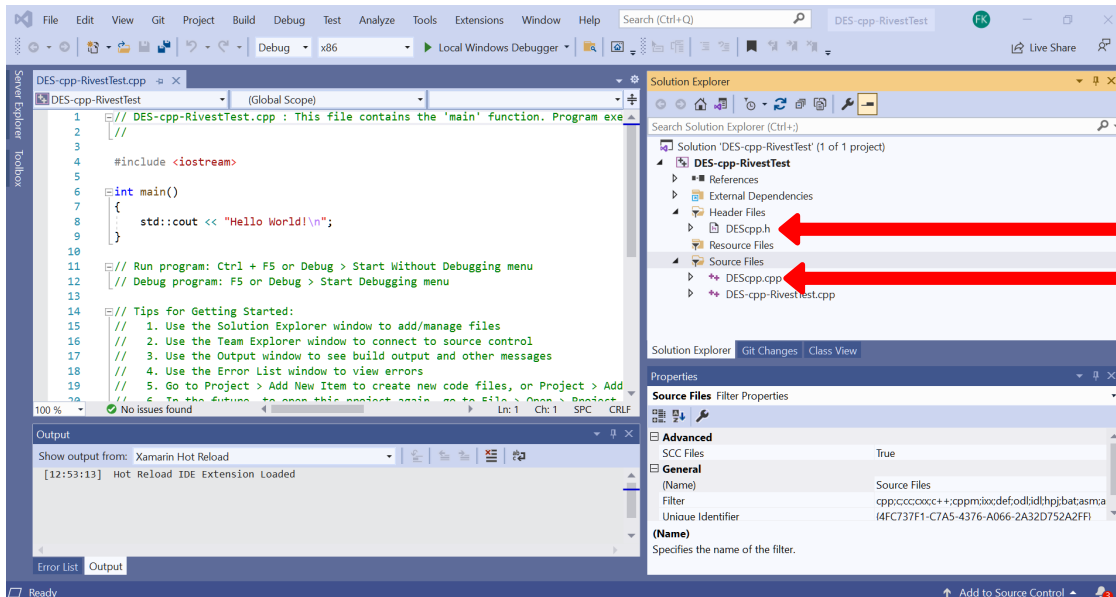
7. In the opening File Window, select `DEScpp.h` and click on Add button.



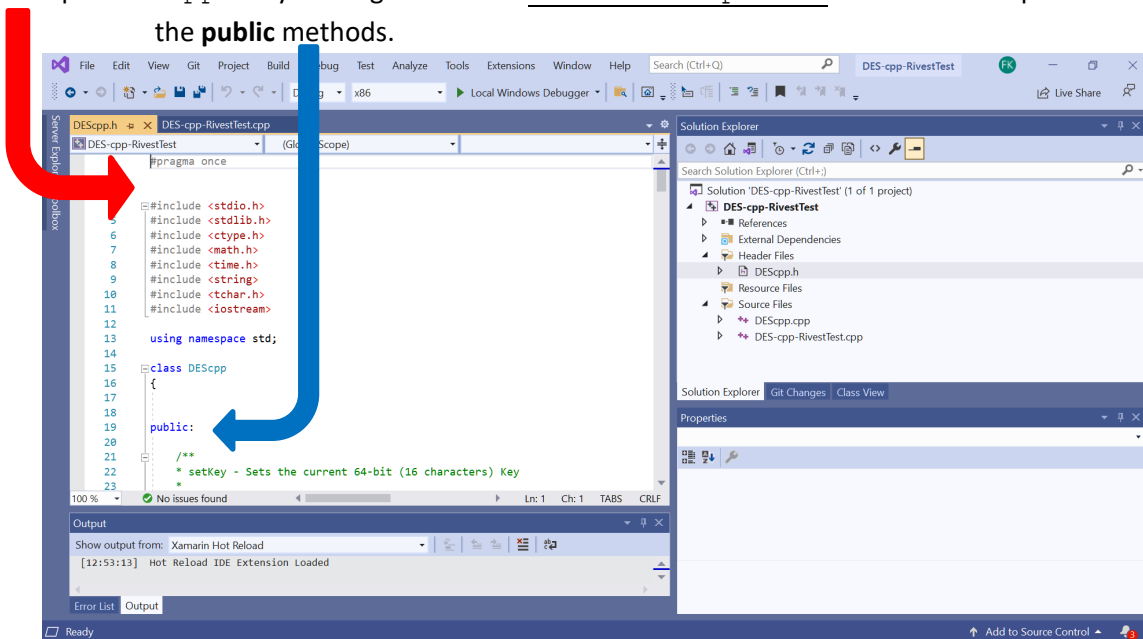
8. Repeat this process, for Source Files at the Solution Explorer window. Now add the existing item, `DEScpp.cpp`.



9. Then this open source code has been included in your project:



10. Open DEScpp.h by clicking on it at the Solution Explorer window and inspect the description of the **public** methods.



Based on the information provided in the DEScpp.h file answers the following questions:

- How many public methods are provided and what are they?
- Why are the only information for the public methods provided? Why any information is not provided for the private methods?
- What is the constraint on the Key?
- How can a plaintext be encrypted using these methods? Provide the steps for encryption.
- How can a ciphertext be decrypted using these methods? Provide the steps for decryption.

LAB EXERCISE - I:

In this section, we are going to learn how we can use DEScpp implementation.

1. Go to `DES-cpp-RivestTest.cpp` file in the Visual Studio. Add `"#include "DEScpp.h" "` and `"using namespace std;"` before the `main()` method. Delete the line automatically created by the Visual Studio: `std::cout << "Hello World!\n";`
2. In the `main()`, create a `DEScpp` object and name it `myDES`.
3. Set the Key to `"0123456789ABCDEF"` and the Plaintext to `"Derby"` using appropriate methods.
TIP: 1. You can use the methods by means of the object you created using `'.'` notation:

```
myDES.<method name>()
```


2. You need to check the error codes when you execute a method that returns an error code and warn the user with the error. You can use the `if` statement for this purpose.
4. Encrypt the Plaintext using an appropriate method and display the Ciphertext.
5. Run the code. **What is the Ciphertext?**
6. Pencil down the Ciphertext. Change your program by setting the Key to `"0123456789ABCDEF"` and the CipherText to the Hex string you recorded by using appropriate methods.
TIP: Delete the lines you used to obtain the Ciphertext.
7. Decrypt the Ciphertext using an appropriate method and display the Plaintext.
8. Run the code. **What is the Plaintext?**

Can this small test show that the implementation is correct? Why?

LAB EXERCISE - II:

In this section, we will perform the Rivest test to check if the implementation is correct.

1. Delete the lines you used to decrypt the Ciphertext.
2. Create a string variable, `X`, and set it to the initial value described in the Tutorial:

```
//Initial String  
string X ="9474B8E8C73BCA7D";
```
3. Using a `for` loop, implement the Rivest Test. **How many steps do we need?**

3.1. In the loop, first, display the loop step and the current value of `X`.

3.2. Check if the step is odd or even.

TIP: You can use `step % 2` operation. If it equals 0 then the step is even, else it is odd. You can use `if ... else` statement.

3.3. According to step is odd or even;

- First set the Key to the current value of `X`.
- Then set the Plaintext **or** the Ciphertext to the current value of `X`.
- Perform encryption **or** decryption.
- Finally, assign the Ciphertext **or** Plaintext to `X`.

TIP: You need to assign HEX value to `X`. Then access the Plaintext using **HEX** format.

What is the last value displayed of `X`?

Is this implementation correct?